Table des matières

Labo 08 — Mettre en place l'intégration continue 1
Avant de commencer 3
Étape 1 — Une cible pour la CI 3
Étape 2 — Le linteur et sa configuration 4
Étape 3 — Ce que `fix` corrige, et ce qu'il abîme 6
Étape 4 — Les cinq écarts qui restent 7
Étape 5 — La fraîcheur, réglée sur la cadence 9
Étape 6 — Le contrat de l'étoile des ventes 11
Étape 7 — La structure vide des sources 12
Étape 8 — Le fond, sur l'entrepôt jetable 13
Étape 9 — Le workflow 14
Étape 10 — Répéter la CI sur une copie neuve 16
Étape 11 — Sur GitHub 17
Étape 12 — La branche qui casse la borne de fin 18
Étape 13 — Nettoyer les schémas de travail 19
Étape 14 — Ce qu'un agent lit du manifeste 20
Étape 15 — La livraison 21
Récapitulatif 21
Dépannage 22
Annexe — La CI sur Snowflake et sur Databricks 23

Labo 08 — Mettre en place l'intégration continue

Labo · Cours 8/9 — Formation Analytics Engineering

Ce que vous allez faire. Mettre le projet sous une vérification qui tourne sans vous. Vous passez le code au linteur, vous réglez la fraîcheur des sources sur leur cadence, vous posez un contrat sur l'étoile des ventes, puis vous montez la CI : une cible à elle, un entrepôt jetable qui n'a que la structure des sources, et un

© 2026 iAlexMG. Tous droits réservés.

workflow que vous répétez d'abord sur votre poste, puis, si vous avez un compte, sur GitHub. Objectif de fin : une branche qui change un seul caractère de `fct_ventes`, refusée par la CI avant que quiconque ait eu à la relire.

Deux terminaux servent dans ce labo, plus un navigateur si vous poussez sur GitHub. L'Anaconda Prompt, environnement `formations` actif, sert à dbt, à sqlfluff, à Git et au petit Python du labo ; il s'ouvre dans `Projet_Boreal/02_ae`, sauf aux étapes 7 et 10, qui disent où aller. La ligne de commande DuckDB, ouverte depuis `Projet_Boreal`, sert à vérifier ce que dbt a écrit, et elle doit être fermée quand dbt travaille.

La liste de contrôle du cours :

- ☐ La cible `ci` déclarée, et son profil admis par le `.gitignore`
- ☐ Le linteur configuré : 46 écarts dans 13 fichiers
- ☐ `sqlfluff fix` passé, et `dim_date` réécrit à la main
- ☐ Les cinq écarts restants traités, le linteur au vert, l'étoile inchangée
- ☐ La fraîcheur réglée sur la cadence de chaque source
- ☐ Le contrat de `fct_ventes`, au vert
- ☐ La structure vide des sources : 27 lignes, 25 tables
- ☐ `dbt build` sur l'entrepôt jetable : `PASS=129` sur 130 nœuds
- ☐ Le contrat mis à l'épreuve, puis rétabli
- ☐ Le workflow écrit, avec sa pile épinglée
- ☐ La CI répétée sur une copie neuve du dépôt
- ☐ (facultatif) La CI au vert sur GitHub
- ☐ La branche qui rend la borne de fin inclusive, refusée, puis réparée
- ☐ Les schémas de travail jetés par `nettoyer_dev`
- ☐ Le `manifest.json` lu comme un agent le lirait
- ☐ `LIVRAISON.md` complété, le travail commité

Le labo se fait en deux séances. La première va jusqu'à l'étape 8, la CI sur votre poste ; la seconde porte le workflow, GitHub, la branche qui casse et le reste.

© 2026 iAlexMG. Tous droits réservés.

Avant de commencer

Ce labo part du projet tel que le Cours 07 l'a laissé, commité. Vérifiez-le dans l'Anaconda Prompt, depuis `Projet_Boreal/02_ae` :

```
dbt build --target prod --exclude snp_concessionnaire
git status --short
```

`dbt build` doit rendre `PASS=128 WARN=0 ERROR=0` sur 129 nœuds, et `git status` ne doit rien afficher. S'il reste des fichiers modifiés, commitez-les d'abord : ce labo retouche une douzaine de modèles d'un coup, et vous voudrez pouvoir comparer.

Le linteur est déjà installé, par le `requirements.txt` de la formation, à l'étape 1 du guide d'installation du parcours :

```
sqlfluff --version
```

Attendu : `sqlfluff, version 4.3.0`. Si la commande est inconnue, installez la pile de la formation depuis le dossier `Data_Domain`, par `pip install -r 02_Analytics_Engineering/requirements.txt`.

Un compte GitHub est facultatif. Sans lui, l'étape 10 répète la CI sur votre poste, ce qui prouve tout ce que le raisonnement demande, et l'étape 11 se lit sans se faire. Avec lui, la ligne de commande de GitHub rend les étapes 11 et 12 plus courtes ; elle s'installe par `winget install --id GitHub.cli --source winget`, puis `gh auth login`, d'après sa documentation consultée le 2026-09-23. Tout ce qu'elle fait se fait aussi dans le navigateur.

Étape 1 — Une cible pour la CI

Théorie : `Theorie_08_AE.md`, sections 6 et 7.

La CI construira dans son propre entrepôt, un fichier DuckDB créé sur place. Elle a donc besoin de son propre profil, et celui-ci vivra dans le dépôt. Créez le dossier `ci`, puis le fichier `ci/profiles.yml` :

```
mkdir ci
notepad ci\profiles.yml
```

```
boreal_ae:
  target: ci
  outputs:
    ci:
      type: duckdb
      path: ci.duckdb
      schema: staging
      threads: 4
```


© 2026 iAlexMG. Tous droits réservés.

Le chemin est relatif, et c'est voulu. Le Cours 03 vous a fait écrire un chemin absolu dans votre profil personnel, parce qu'un chemin relatif se résout depuis le dossier où l'on lance dbt. Ici, c'est ce qu'on veut : la CI se lance toujours depuis la racine du dépôt, et un chemin absolu désignerait un dossier de votre poste que la machine de GitHub n'a pas.

Faites-le entrer dans l'index :

```
git add ci/profiles.yml
```

Git refuse :

```
The following paths are ignored by one of your .gitignore files:
ci/profiles.yml
hint: Use -f if you really want to add them.
```

Demandez-lui quelle ligne en est la cause, puis posez la même question pour le fichier d'entrepôt que la CI créera :

```
git check-ignore -v ci/profiles.yml
git check-ignore -v ci.duckdb
```

La première commande rend `.gitignore:8:profiles.yml  ci/profiles.yml` : la ligne du Cours 01, qui visait votre fichier de connexion, vise tout fichier de ce nom, où qu'il soit. La seconde ne rend rien : `ci.duckdb` n'est pas ignoré, et il entrerait au premier `git add .`. Le numéro de ligne peut différer si votre `.gitignore` n'est pas celui du Labo 01 au caractère près.

Ajoutez trois lignes à la fin du `.gitignore`, sous un commentaire qui dit pourquoi :

```
# La CI : son profil ne porte aucune valeur secrète, et il se versionne ; son entrepôt, non
!ci/profiles.yml
ci.duckdb
ci.duckdb.wal
```

Le point d'exclamation fait une exception, et elle ne vaut que si elle suit la règle qu'elle contredit. Reposez les deux questions : Git répond maintenant `.gitignore:12:!ci/profiles.yml` pour le profil, et `.gitignore:13:ci.duckdb` pour l'entrepôt. `git add ci/profiles.yml` passe.

Pourquoi — ce `profiles.yml`-là se versionne, et le vôtre jamais. Celui de votre poste porte un chemin de votre machine, et porterait un compte et un secret sur une plateforme. Celui de la CI ne porte qu'un nom de fichier relatif, rien qui vaille d'être protégé. L'annexe montre ce qu'il devient sur Snowflake ou Databricks : des appels `env_var(...)`, des noms de variables et aucune valeur.

Étape 2 — Le linteur et sa configuration

Théorie : `Theorie_08_AE.md`, section 4.

sqlfluff lit sa configuration dans un fichier `.sqlfluff` posé à la racine du dépôt. Créez-le :

© 2026 iAlexMG. Tous droits réservés.

```
[sqlfluff]
dialect = duckdb
templater = dbt
max_line_length = 100
exclude_rules = ST06, ST09

[sqlfluff:templater:dbt]
project_dir = .
profiles_dir = ci
target = ci

[sqlfluff:indentation]
tab_space_size = 4
template_blocks_indent = false

[sqlfluff:rules:capitalisation.keywords]
capitalisation_policy = lower

[sqlfluff:rules:capitalisation.identifiers]
extended_capitalisation_policy = lower

[sqlfluff:rules:capitalisation.functions]
extended_capitalisation_policy = lower

[sqlfluff:rules:capitalisation.literals]
capitalisation_policy = lower

[sqlfluff:rules:capitalisation.types]
extended_capitalisation_policy = lower
```

Quatre blocs, quatre décisions.

La section `[sqlfluff]` dit le dialecte, DuckDB, et le templater, dbt : le linteur compilera le projet avant de lire, Jinja comprise. Elle écarte deux règles, et la section 4 de la théorie dit pourquoi : `ST06` réordonnerait les colonnes des modèles, `ST09` retournerait les jointures, et aucune des deux ne corrige un défaut. Écrivez la raison en commentaire au-dessus de la ligne, comme le fait le `Solution/` : dans un an, c'est la seule chose qui empêchera quelqu'un de les rallumer.

La section `[sqlfluff:templater:dbt]` dit où trouver le profil et quelle cible compiler. C'est la seule place où ces clés se lisent. Un `_dir` s'y résout depuis le `.sqlfluff`, donc `ci` désigne le dossier de l'étape 1 où que vous lanciez la commande.

`template_blocks_indent = false` empêche le correcteur de réindenter le contenu des blocs Jinja, et le snapshot en a besoin : l'étape 3 dit ce qui arrive sans.

Les cinq sections de casse imposent les minuscules partout, la convention de l'écosystème dbt et celle de toute la formation.

© 2026 iAlexMG. Tous droits réservés.

Lancez le linteur sur les trois dossiers qui portent du SQL :

```
sqlfluff lint models tests snapshots
```

Comptez une quinzaine de secondes : le templater compile le projet d'abord. Attendu : 13 fichiers en `FAIL`, pour 46 écarts.

Le code	Ce qu'il signale	Écarts
`LT02`	une indentation qui ne suit pas la structure	38
`RF02`	une colonne sans préfixe dans une requête qui lit plusieurs tables	3
`RF01`	une référence à une table introuvable	2
`LT09`	plusieurs colonnes sur la ligne du `select`	2
`LT14`	un `where` qui ne commence pas sa ligne	1

Remarquez aussi qu'un fichier `ci.duckdb` est apparu à la racine du dépôt : pour compiler, le templater a ouvert l'entrepôt de la cible `ci`, qui n'existait pas. C'est la ligne ajoutée à l'étape 1 qui le garde hors du dépôt.

Piège — l'option qui n'existe pas. Le réflexe, quand le linteur ne trouve pas le profil, est de lui passer l'option de dbt : `sqlfluff lint --profiles-dir ci` rend `Error: No such option '--profiles-dir'.` et le code 2. L'autre réflexe, ranger `profiles_dir` sous `[sqlfluff]`, est pire : la clé est ignorée sans un mot, et le linteur retombe sur votre profil personnel. Si `target` a suivi, tout passe au vert sur votre poste et casse sur la machine de GitHub, qui n'a pas ce profil, sur « Could not find profile named 'boreal_ae' ».

Étape 3 — Ce que `fix` corrige, et ce qu'il abîme

Théorie : `Theorie_08_AE.md`, section 4.

Le linteur sait corriger une partie de ce qu'il signale. Lancez-le en mode correction :

```
sqlfluff fix models tests snapshots
```

Attendu, en une quinzaine de secondes : douze fichiers marqués `FIXED`, et le bilan `41 fixable linting violations found`, suivi de `[5 unfixable linting violations found]`.

Regardez ce qui a bougé avant de le croire :

```
git diff --stat
```


© 2026 iAlexMG. Tous droits réservés.

Douze fichiers, 58 lignes ajoutées et 36 retirées. Git affiche aussi, fichier par fichier, que les fins de ligne passeront en CRLF : le correcteur écrit en fins de ligne Unix, et Git les normalise sous Windows au commit, sans effet sur le contenu.

La plupart des corrections sont celles qu'on aurait faites à la main. La jointure temporelle de `fct_ventes`, par exemple, a pris la forme que le linteur impose à une condition sur plusieurs lignes :

```
  inner join {{ ref('dim_concessionnaire') }} as d
      on
          d.code_concessionnaire = l.code_concessionnaire
          and l.date_commande >= d.valide_de
          and (d.valide_jusqua is null or l.date_commande < d.valide_jusqua)
```

Ouvrez maintenant `models/marts/dim_date.sql`. La liste des noms de mois en est sortie ainsi :

```
      [
          'janvier', 'février', 'mars', 'avril', 'mai', 'juin',
          'juillet', 'août', 'septembre', 'octobre', 'novembre',
          'décembre'
      ][month(date_day)
      ] as nom_mois,
```

Conforme et illisible, et la liste des jours a subi le même sort. Réécrivez les deux à la main, la liste sur ses propres lignes et l'indice collé au crochet :

```
      [
          'janvier', 'février', 'mars', 'avril', 'mai', 'juin',
          'juillet', 'août', 'septembre', 'octobre', 'novembre', 'décembre'
      ][month(date_day)] as nom_mois,
      [
          'lundi', 'mardi', 'mercredi', 'jeudi', 'vendredi', 'samedi', 'dimanche'
      ][isodow(date_day)] as nom_jour,
```

Piège — le correcteur et la Jinja. `sqlfluff fix` ne voit que la branche d'un `{% if %}` que la compilation a prise. Sans `template_blocks_indent = false`, il trouve seize écarts dans le seul snapshot, en corrige quinze, indente tout son corps d'un cran, et laisse la branche non prise collée à la marge : le fichier sort à moitié décalé. Si c'est ce que vous voyez, le réglage manque à votre `.sqlfluff`.

Étape 4 — Les cinq écarts qui restent

Théorie : `Theorie_08_AE.md`, section 4.

Relancez le linteur :

```
  sqlfluff lint models tests snapshots
```


© 2026 iAlexMG. Tous droits réservés.

Trois fichiers restent en `FAIL`. Deux signalent une référence non qualifiée dans une sous-requête :

```
== [models\intermediate\int_usine_par_actif.sql] FAIL
L:  42 | P:  16 | RF02 | Unqualified reference 'actif_id' found in select with
                       | more than one referenced table/view.
L:  43 | P:  15 | RF02 | Unqualified reference 'grandeur' found in select with
                       | more than one referenced table/view.
```

Ce n'est pas une affaire de style. Dans la ligne de commande DuckDB, depuis `Projet_Boreal` :

```
select count(*) from marts.fct_ventes
where cle_produit in (select cle_produit from marts.dim_date);
```

Attendu : 80 618, toutes les lignes du fait. `dim_date` n'a pas de colonne `cle_produit`, et la requête ne s'en plaint pas : une colonne sans préfixe se cherche dans la sous-requête, puis dans la requête qui l'entoure, et chaque ligne du fait se compare à elle-même. Un préfixe l'aurait fait échouer sur le champ. Fermez DuckDB par `.exit`.

Qualifiez donc la sous-requête de `int_usine_par_actif`, par un alias :

```
where
    actif_id in (
        select m.actif_id from {{ ref('stg_etiquettes') }} as m
        where m.grandeur = 'MARCHE'
    )
```

Et celle du snapshot, dans `snapshots/snp_concessionnaire.sql`, qui lit le lot le plus récent :

```
where _lot = (select max(d._lot) from {{ source('raw', 'erp_kna1') }} as d)
```

Le troisième fichier est `int_jalons_reclamations`, et ses deux écarts sont de faux positifs :

```
L:  18 | P:   9 | RF01 | Reference 'etape.statut' refers to table/view not found
                       | in the FROM clause or found in ancestor statement.
```

`etape` n'est pas une table : c'est la colonne de type structure que le latéral du Cours 06 déplie, et `etape.statut` en lit un champ. Dites-le au linteur, ligne par ligne :

```
        etape.statut,  -- noqa: RF01
        etape.date  -- noqa: RF01
```

Un `-- noqa` sans code ferait taire toutes les règles sur la ligne ; avec le code, il ne fait taire que celle qu'on a regardée.

Relancez le linteur. Attendu : aucun `FAIL`, et `All Finished!`. Puis prouvez que rien n'a changé dans ce que le projet produit :

```
dbt build --target prod --exclude snp_concessionnaire
```


© 2026 iAlexMG. Tous droits réservés.

`PASS=128 WARN=0 ERROR=0`, comme avant le linteur. Et dans DuckDB :

```
select
    (select count(*) from marts.fct_ventes) as ventes,
    (select sum(valeur_nette) from marts.fct_ventes) as valeur_nette,
    (select count(*) from marts.fct_reclamations) as reclamations,
    (select count(*) from marts.fct_sorties) as sorties,
    (select count(*) from marts.dim_date) as jours;
```

Attendu : 80 618 lignes, 465 924 613,14 $, 4 337 réclamations, 201 542 sorties et 1 551 jours, les chiffres du Cours 07. Le linteur a touché treize fichiers, et pas un nombre.

Étape 5 — La fraîcheur, réglée sur la cadence

Théorie : `Theorie_08_AE.md`, section 8.

Ouvrez `models/staging/_sources.yml`. Sous le bloc `raw`, après `schema: raw`, ajoutez la colonne de chargement et les seuils des sources mensuelles :

```
    config:
      loaded_at_field: _charge_le
      freshness:
        warn_after: {count: 35, period: day}
        error_after: {count: 40, period: day}
```

Le réglage du bloc vaut pour toutes ses tables, et une table qui a une autre cadence le redit chez elle. La télémétrie arrive chaque nuit :

```
      - name: telemetrie_sorties
        description: "Un message par sortie d'un véhicule connecté, objet JSON."
        config:
          freshness:
            warn_after: {count: 36, period: hour}
            error_after: {count: 72, period: hour}
```

Et les deux exports de l’usine n’ont pas de cadence : ce sont des listes complètes, livrées une fois, dont l’âge ne dit rien.

```
      - name: usine_actifs
        description: "Le registre des actifs de l'usine : six zones et dix-sept équipements."
        config:
          freshness: null
      - name: usine_etiquettes
        description: "Les 58 points mesurés de l'historien, rattachés à leur actif."
        config:
          freshness: null
```


© 2026 iAlexMG. Tous droits réservés.

Le bloc `argent` ne reçoit rien : ses tables n'ont pas de colonne de chargement, et leur fraîcheur se surveille chez Data Engineering. Écrivez ces raisons dans l'en-tête du fichier, à côté des chiffres : un seuil sans sa raison est un seuil que personne n'osera toucher.

Avant de lancer la commande, prévoyez son verdict. Dans DuckDB, depuis `Projet_Boreal` :

```
select 'erp_vbak' as source, max(_charge_le) as dernier_chargement,
       round(date_diff('minute', max(_charge_le), now()) / 60, 1) as heures
from raw.erp_vbak
union all
select 'garanties_reclamations', max(_charge_le),
       round(date_diff('minute', max(_charge_le), now()) / 60, 1)
from raw.garanties_reclamations
union all
select 'telemetrie_sorties', max(_charge_le),
       round(date_diff('minute', max(_charge_le), now()) / 60, 1)
from raw.telemetrie_sorties;
```

Les trois sources portent la date de votre dernier chargement, celui de l'étape 7 du Labo 07. Fermez DuckDB, puis :

```
dbt source freshness --target prod
```

Dix sources sont vérifiées, celles qui ont une colonne de chargement et un seuil. Le verdict dépend de l'âge que vous venez de lire, et c'est tout l'intérêt de l'exercice.

L'âge de votre dernier chargement	Les neuf sources mensuelles	La télémétrie	Le code de sortie
moins de 36 heures	`PASS`	`PASS`	0
de 36 à 72 heures	`PASS`	`WARN`	0
de 72 heures à 35 jours	`PASS`	`ERROR STALE`	1
de 35 à 40 jours	`WARN`	`ERROR STALE`	1

Mesuré le jour même du chargement : 10 `PASS` et le code 0. Si vous faites ce labo quelques jours après le Labo 07, la télémétrie sort en `ERROR STALE`, et la commande a raison : votre poste ne reçoit pas d'archive chaque nuit, et la source est vraiment périmée. Les sources mensuelles, elles, restent vertes pendant tout le mois qui suit leur livraison, ce que des seuils ronds de 72 heures ne permettraient pas : la théorie mesure dix erreurs sur une copie où huit sources étaient à jour.

À retenir — la deuxième ligne du tableau rend 0. Un avertissement seul ne fait pas échouer la commande, et une étape de CI qui rend 0 passe au vert. Ce qui doit arrêter une tâche est une erreur ; un avertissement ne sert que si quelqu'un lit le journal.

© 2026 iAlexMG. Tous droits réservés.

Étape 6 — Le contrat de l'étoile des ventes

Théorie : `Theorie_08_AE.md`, section 9.

`fct_ventes` est lu par Business Intelligence, et l'`exposure` du Cours 05 le déclare. Son interface mérite un contrat. Dans `models/marts/_marts.yml`, sous `fct_ventes`, ajoutez la configuration du contrat, puis donnez à chacune des neuf colonnes son type, dans l'ordre où le modèle les rend :

```yaml
  - name: fct_ventes
    description: "Le fait des ventes, au grain de la ligne de commande, hors postes refusés."
    config:
      contract:
        enforced: true
    data_tests:
      - dbt_utils.unique_combination_of_columns:
          arguments:
            combination_of_columns:
              - numero_commande
              - numero_poste
    columns:
      - name: cle_date
        data_type: date
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_date')
                field: date_jour
      - name: cle_concessionnaire
        data_type: varchar
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_concessionnaire')
                field: cle_concessionnaire
      - name: cle_produit
        data_type: varchar
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_produit')
                field: cle_produit
```


© 2026 iAlexMG. Tous droits réservés.

```
      - name: numero_commande
        data_type: varchar
      - name: numero_poste
        data_type: varchar
      - name: quantite
        data_type: decimal(18, 3)
      - name: valeur_nette
        data_type: decimal(18, 2)
        data_tests:
          - not_null
      - name: cout
        data_type: decimal(18, 2)
      - name: marge
        data_type: decimal(18, 2)
```

Un contrat veut toutes les colonnes, pas seulement celles qu'un test surveille : c'est pour cela que `numero_commande`, `numero_poste`, `quantite`, `cout` et `marge` entrent dans la liste. Et `cle_concessionnaire` remonte à sa place, juste après `cle_date`, parce que le contrat se lit comme la sortie du modèle.

```
dbt build --target prod --exclude snp_concessionnaire
```

Attendu : `PASS=128 WARN=0 ERROR=0`. Le contrat n'ajoute aucun nœud, il ajoute une vérification à la construction du modèle, et le modèle la passe. Vous le mettrez à l'épreuve à l'étape 8, là où il sert.

Commitez ce premier état : les étapes suivantes font des essais qu'on défait, et il vaut mieux les défaire contre un commit que de mémoire.

```
git add .gitignore .sqlfluff ci models snapshots tests
git commit -m "Cours 08 : le linteur, la fraicheur et le contrat"
```

Étape 7 — La structure vide des sources

Théorie : `Theorie_08_AE.md`, section 5.

La CI n'aura pas les données, seulement leur forme. Tirez-la de l'entrepôt, dans l'Anaconda Prompt, depuis `Projet_Boreal`, en une seule ligne :

```
duckdb -readonly plateforme/entrepot/boreal.duckdb -noheader -list -c "select 'create schema ' || schema_name || ';' from duckdb_schemas() where schema_name in ('raw', 'argent') order by 1; select sql from duckdb_tables() where schema_name in ('raw', 'argent') order by schema_name, table_name;" > 02_ae/ci/structure_sources.sql
```

La première requête écrit la création des deux schémas, la seconde celle de chaque table, que DuckDB garde toute prête dans son catalogue : la colonne `sql` de `duckdb_tables()` porte l'instruction et son point-virgule. `-noheader -list` retire l'en-tête et le cadre, pour qu'il ne reste que du SQL.

Ouvrez le fichier. Attendu : 27 lignes, deux `create schema` et 25 `CREATE TABLE`, trois dans `argent` et vingt-deux dans `raw`. Les premières :

© 2026 iAlexMG. Tous droits réservés.

```
create schema argent;
create schema raw;
CREATE TABLE argent.energie_horaire(heure TIMESTAMP WITH TIME ZONE, energie_kwh DOUBLE, energie_kvarh DOUBLE, releves BIGINT,
releves_estimes BIGINT);
```

Revenez dans `Projet_Boreal/02_ae`. Supprimez le `ci.duckdb` que le linteur a créé à l'étape 2, puis montez la structure dans un entrepôt neuf, comme le fera la CI :

```
del ci.duckdb
python -c "import duckdb; duckdb.connect('ci.duckdb').execute(open('ci/structure_sources.sql', encoding='utf-8').read())"
python -c "import duckdb; print(duckdb.connect('ci.duckdb', read_only=True).sql('select schema_name, count(*) from
duckdb_tables() group by 1 order by 1').fetchall())"
```

Attendu : `[('argent', 3), ('raw', 22)]`, en un cinquième de seconde. Vingt-cinq tables, et pas une ligne.

Étape 8 — Le fond, sur l'entrepôt jetable

Théorie : `Theorie_08_AE.md`, sections 5, 6 et 9.

Construisez le projet sur la cible de la CI, en disant à dbt où trouver son profil :

```
dbt build --target ci --profiles-dir ci
```

Attendu, en une dizaine de secondes : `PASS=129 WARN=0 ERROR=0 SKIP=0 NO-OP=1`, sur 130 nœuds. Le `NO-OP` est l'`exposure`, qui n'a rien à exécuter. Le snapshot n'est pas exclu, contrairement au poste : sur un entrepôt neuf, il n'a aucune histoire à abîmer, et `dim_concessionnaire` a besoin que sa table existe.

En tête de l'exécution, dbt affiche un avertissement sur le snapshot, `Data type of snapshot table timestamp columns (TIMESTAMP) doesn't match derived column 'updated_at' (DATE)`. Vous l'avez déjà vu à chaque lot du snapshot depuis le Cours 05, et il ne décrit pas la table : ses colonnes de validité sont en `DATE`, vérifié sur l'entrepôt du poste comme sur celui de la CI. Il n'empêche rien.

Regardez ce que la construction a produit :

```
python -c "import duckdb; c = duckdb.connect('ci.duckdb', read_only=True); print(c.sql(\"select table_schema, table_type,
count(*) from information_schema.tables where table_schema not in ('raw', 'argent') group by all order by 1\").fetchall())"
```

Trente objets : quinze tables dans `dev_marts`, treize vues dans `dev_staging`, une vue dans `dev_presentation` et le snapshot. Les schémas portent le préfixe `dev_`, puisque la cible n'est pas `prod`. Trois objets seulement ont des lignes : le seed des anomalies et `dim_anomalie`, 5 chacun, et `dim_date`, 1 551 jours. Tout le reste est vide, et les 98 tests de données sont passés au vert sans rien vérifier.

Mettez maintenant le contrat à l'épreuve. Dans `models/marts/fct_ventes.sql`, remplacez la ligne `l.quantite,` par :

```
    l.quantite::integer as quantite,
```

```
dbt build --target ci --profiles-dir ci
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : `PASS=117 WARN=0 ERROR=1 SKIP=12`, et le contrat qui refuse de construire, avec son tableau :

```
This model has an enforced contract that failed.
| column_name | definition_type | contract_type | mismatch_reason    |
| ----------- | --------------- | ------------- | ------------------ |
| quantite    | INTEGER         | DECIMAL(18,3) | data type mismatch |
```

Voyez ce qui se passerait sans lui. Retirez les trois lignes du contrat dans `_marts.yml`, `config:`, `contract:` et `enforced: true`, puis :

```
dbt build --target ci --profiles-dir ci --select fct_ventes
python -c "import duckdb; print(duckdb.connect('ci.duckdb', read_only=True).sql(\"select data_type from
information_schema.columns where table_name = 'fct_ventes' and column_name = 'quantite'\").fetchall())"
```

Attendu : `PASS=11`, et `[('INTEGER',)]`. Le modèle passe, ses tests aussi, et l'interface a changé en silence. Sur l'entrepôt du poste, aucun test de données ne l'aurait vu non plus : les 80 618 quantités de Boréal sont entières. Défaites les deux essais contre le commit de l'étape 6 :

```
git restore models/marts/fct_ventes.sql models/marts/_marts.yml
```

Étape 9 — Le workflow

Théorie : `Theorie_08_AE.md`, section 3.

La machine de GitHub installera la pile à chaque exécution, et elle doit installer exactement celle de votre poste. Créez `ci/requirements.txt` :

```
dbt-core==1.12.3
dbt-duckdb==1.11.0
duckdb==1.5.5
sqlfluff==4.3.0
sqlfluff-templater-dbt==4.3.0
```

Des signes `==`, et non les fourchettes du `requirements.txt` de la formation : celui-ci installe un poste, celui-là fabrique une preuve, et une preuve ne doit pas changer de version un matin sans que personne ait touché au code. Écrivez-le en commentaire en tête du fichier.

Puis le workflow. GitHub le cherche dans un dossier précis :

```
mkdir .github\workflows
notepad .github\workflows\ci.yml
```

```
name: CI dbt

on:
  pull_request:
    branches: [main]
  push:
```


© 2026 iAlexMG. Tous droits réservés.

```yaml
    branches: [main]

# Le jeton que GitHub prete a la tache ne peut que lire le depot.
permissions:
  contents: read

jobs:
  verifier:
    runs-on: ubuntu-latest
    timeout-minutes: 15

    steps:
      - name: Récupérer le dépôt
        uses: actions/checkout@v7

      - name: Installer Python 3.11
        uses: actions/setup-python@v7
        with:
          python-version: "3.11"
          cache: pip
          cache-dependency-path: ci/requirements.txt

      - name: Installer la pile, aux versions du poste
        run: pip install -r ci/requirements.txt

      # Avant sqlfluff : son templater compile le projet, et le projet appelle dbt_utils.
      - name: Installer dbt_utils
        run: dbt deps

      # Le profil de la CI se declare dans .sqlfluff, sous [sqlfluff:templater:dbt] :
      # sqlfluff ne connait pas l'option --profiles-dir.
      - name: La forme, par sqlfluff
        run: sqlfluff lint models tests snapshots

      - name: Monter la structure vide des sources
        run: python -c "import duckdb; duckdb.connect('ci.duckdb').execute(open('ci/structure_sources.sql', encoding='utf-
8').read())"

      # Le snapshot n'est pas exclu ici, contrairement au poste : sur un entrepot neuf et vide, il
      # n'a aucune histoire a abimer, et dim_concessionnaire a besoin que sa table existe.
      - name: Le fond, par dbt build
        run: dbt build --target ci --profiles-dir ci
```


© 2026 iAlexMG. Tous droits réservés.

```yaml
      - name: Générer la documentation
        run: dbt docs generate --target ci --profiles-dir ci

      - name: Publier la documentation
        uses: actions/upload-artifact@v7
        with:
          name: documentation-dbt
          path: |
            target/index.html
            target/manifest.json
            target/catalog.json
```

La section 3 de la théorie le lit ligne à ligne. Trois points à vérifier avant d'enregistrer, parce que ce sont eux qui cassent : l'indentation est faite d'espaces, jamais de tabulations, YAML refusant les secondes ; `dbt deps` précède le linteur ; et chaque commande dbt de la CI porte `--profiles-dir ci`, pendant que le linteur n'en porte aucune, puisqu'il lit le sien dans `.sqlfluff`.

Le `Solution/` porte en tête du fichier un bloc de commentaires qui dit ce que le workflow vérifie et ce qu'il ne vérifie pas. Écrivez le vôtre : c'est la première chose que lira celui qui verra la CI tomber.

Étape 10 — Répéter la CI sur une copie neuve

Théorie : `Theorie_08_AE.md`, sections 3 et 5.

Avant GitHub, la répétition. Le runner partira d'une copie du dépôt, sans rien de ce qui traîne sur votre poste. Faites de même : commitez, puis clonez le dépôt à côté de lui.

```
git add ci .github
git commit -m "Cours 08 : la CI"
cd ..
git clone 02_ae 02_ae_ci
cd 02_ae_ci
```

La copie n'a ni `target/`, ni `dbt_packages/`, ni `ci.duckdb` : seulement ce que Git suit. Lancez les étapes du workflow dans l'ordre, en commençant par une inversion volontaire, le linteur avant `dbt deps` :

```
sqlfluff lint models tests snapshots
```

Il s'arrête avant d'avoir lu une ligne :

```
User Error: dbt failed during project compilation. ... UninstalledPackagesFoundError: Compilation Error
  dbt expects 1 package(s) based on packages specified in packages.yml, but found only 0 package(s)
```


© 2026 iAlexMG. Tous droits réservés.

```
    installed in dbt_packages. Following packages were not found: dbt_utils. Run "dbt deps" to
    install package dependencies.
```

C'est la raison de l'ordre du workflow. Maintenant, dans le bon :

```
dbt deps
sqlfluff lint models tests snapshots
python -c "import duckdb; duckdb.connect('ci.duckdb').execute(open('ci/structure_sources.sql', encoding='utf-8').read())"
dbt build --target ci --profiles-dir ci
dbt docs generate --target ci --profiles-dir ci
```

Attendu : `dbt_utils` installé en 1.4.1 ; `All Finished!` au linteur, en une quinzaine de secondes ; `PASS=129` au `dbt build` ; et trois fichiers dans `target/`, `index.html`, `manifest.json` et `catalog.json`, ce que l'artefact publiera. Votre profil personnel existe toujours, et aucune de ces commandes ne l'a lu : c'est ce qui rend la répétition fidèle.

```
git status --short
```

Rien : tout ce que la CI a fabriqué est ignoré. Revenez au dépôt et jetez la copie :

```
cd ..
rmdir /s /q 02_ae_ci
cd 02_ae
```

Étape 11 — Sur GitHub

Théorie : `Theorie_08_AE.md`, sections 2 et 3.

Cette étape demande un compte GitHub. Sans lui, lisez-la : ce qu'elle montre, l'étape 10 l'a déjà prouvé sur votre poste.

Vérifiez d'abord qu'aucun chemin de votre poste n'est entré dans le dépôt :

```
git grep -n -i "users"
```

Rien ne doit sortir. Créez ensuite sur GitHub un dépôt privé et vide, sans `README` ni `.gitignore`, par exemple `boreal-02-ae`, puis reliez-le et poussez :

```
git remote add origin https://github.com/<vous>/boreal-02-ae.git
git push -u origin main
```

La poussée sur `main` déclenche le workflow. Suivez-le dans l'onglet Actions du dépôt, ou par :

```
gh run watch
```

Attendu : la tâche `verifier` au vert en moins d'une minute, 55 secondes mesurées, avec toutes ses étapes cochées, et un artefact `documentation-dbt` d'environ 650 Ko en bas de la page de l'exécution. Téléchargez-le : c'est le site de la documentation, qui ne vit plus seulement sur votre poste.

© 2026 iAlexMG. Tous droits réservés.

Étape 12 — La branche qui casse la borne de fin

Théorie : `Theorie_08_AE.md`, sections 1, 2 et 5.

Proposez un changement comme on le fait à plusieurs, sur une branche :

```
git switch -c borne-de-fin
```

Dans `models/marts/fct_ventes.sql`, rendez la borne de fin inclusive, un seul caractère :

```
        and (d.valide_jusqua is null or l.date_commande <= d.valide_jusqua)
```

Relisez-la comme le ferait un collègue pressé : elle a l'air juste, et même prudente. Demandez l'avis de la CI, sur votre poste :

```
dbt build --target ci --profiles-dir ci
```

Attendu : `PASS=116 WARN=0 ERROR=1 SKIP=13`, et le test unitaire du Cours 05 qui tombe :

```
[ERROR]: in unit_test test_version_d_epoque (models\marts\_marts.yml)
actual differs from expected:
@@ ,numero_commande,cle_concessionnaire
   ,C1             ,ancienne
+++,C2             ,ancienne
   ,C2             ,neuve
```

La vente `C2` tombe le jour même du changement de version. Avec la borne inclusive, elle trouve les deux versions, et le test voit une ligne de trop, marquée `+++`. Sur des tables vides, c'est le seul test qui pouvait le voir : il a apporté ses propres lignes. dbt saute ensuite `fct_ventes` et tout ce qui en dépend, treize nœuds.

Voyez ce que diraient les tests de données, sur les vraies données. Construisez le projet sur la cible `dev`, sans les tests, puis testez le fait :

```
dbt seed
dbt run
dbt test --select fct_ventes
```

Attendu : `PASS=28` au `dbt run`, puis `PASS=7 WARN=0 ERROR=3` au test : le test unitaire, le grain sur 16 couples et la réconciliation sur 4 commandes. Les tests de données le voient aussi, mais seulement parce que 56 ventes tombent le 2024-07-31, le jour où 49 concessionnaires ont changé de version. Un défaut de borne sur une date où rien ne se vend passerait au travers ; le test unitaire, lui, ne dépend pas du hasard du calendrier.

Si vous avez poussé à l'étape 11, proposez le changement pour de vrai :

```
git commit -am "fct_ventes : la borne de fin devient inclusive"
git push -u origin borne-de-fin
gh pr create --fill
```


© 2026 iAlexMG. Tous droits réservés.

Ou ouvrez la demande de tirage dans le navigateur, depuis le bandeau que GitHub affiche après la poussée. La CI tourne sur la demande de tirage et passe au rouge à l'étape « Le fond, par dbt build », avec le même tableau ; les deux étapes de la documentation sont sautées. La demande affiche un échec, et quiconque la relit le voit avant d'avoir lu une ligne de SQL.

Réparez sur la branche, par un commit qui défait le précédent :

```
git revert --no-edit HEAD
git push
```

La CI repart et repasse au vert, en 50 secondes mesurées. Il n'y a plus rien à fusionner : fermez la demande de tirage sans la fusionner, puis nettoyez.

```
git switch main
git branch -D borne-de-fin
git push origin --delete borne-de-fin
```

Sans compte GitHub, remettez le < à la main, puis `git switch main` et `git branch -D borne-de-fin` : les deux premiers paragraphes de l'étape ont montré l'essentiel.

Étape 13 — Nettoyer les schémas de travail

Théorie : `Theorie_08_AE.md`, section 6.

L'étape 12 a rempli les schémas `dev_*` d'un projet cassé, et le Cours 07 vous avait promis une macro pour les jeter. Créez `macros/nettoyer_dev.sql` :

```
{% macro nettoyer_dev() %}
    {% if target.name == 'prod' %}
        {{ exceptions.raise_compiler_error(
            "nettoyer_dev refuse la cible prod : elle ne jette que des schemas de travail."
        ) }}
    {% endif %}
    {% for dossier in ['staging', 'marts', 'presentation'] %}
        {% set schema = generate_schema_name(dossier, none) | trim %}
        {% do run_query('drop schema if exists ' ~ schema ~ ' cascade') %}
        {{ log('Schema jete : ' ~ schema, info=True) }}
    {% endfor %}
{% endmacro %}
```

Deux choix la rendent sûre. Elle ne nomme aucun schéma en dur : elle demande chaque nom à `generate_schema_name`, la macro qui l'a fabriqué, et suit donc la convention le jour où elle change. Et elle commence par son verrou. Un fichier de macro commence par un en-tête `{# … #}` qui dit cela, comme au Cours 05.

Une macro s'exécute hors de toute construction par `dbt run-operation` :

```
dbt run-operation nettoyer_dev
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : trois lignes, `Schema jete : dev_staging`, `dev_marts` et `dev_presentation`. Essayez le verrou :

```
dbt run-operation nettoyer_dev --target prod
```

Attendu : `nettoyer_dev refuse la cible prod : elle ne jette que des schemas de travail.`, et le code 1. Rien n'a été touché. Vérifiez dans DuckDB :

```
select schema_name from duckdb_schemas() where database_name = 'boreal' order by 1;
```

Attendu : `argent`, `main`, `marts`, `pipeline`, `presentation`, `raw`, `snapshots` et `staging`. Plus aucun `dev_*`, et le schéma `snapshots`, qui n'est pas dans la liste de la macro, est intact.

Étape 14 — Ce qu'un agent lit du manifeste

Théorie : `Theorie_08_AE.md`, section 10.

Un agent n'a pas besoin de votre écran pour savoir ce que l'entrepôt contient : il lit le `manifest.json`. Faites comme lui. Créez `outils/lire_manifeste.py` :

```
import json

m = json.load(open("target/manifest.json", encoding="utf-8"))
noeuds = m["nodes"]

types = {}
for n in noeuds.values():
    types[n["resource_type"]] = types.get(n["resource_type"], 0) + 1
print(types, len(m["sources"]), "sources", len(m["exposures"]), "exposure")

v = noeuds["model.boreal_ae.fct_ventes"]
print(v["description"])
print(v["config"]["materialized"], v["config"]["contract"])
for c in v["columns"].values():
    print(" ", c["name"], c.get("data_type"))
print("lit :", v["depends_on"]["nodes"])
enfants = m["child_map"]["model.boreal_ae.fct_ventes"]
print("servi a :", len(enfants), "noeuds")
for e in enfants:
    print(" ", e)
```

Le manifeste s'écrit à chaque commande qui lit le projet. `dbt parse` suffit à le rafraîchir, sans rien construire :

```
dbt parse
python outils/lire_manifeste.py
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : `{'model': 31, 'snapshot': 1, 'test': 98, 'seed': 1} 14 sources 1 exposure`, puis la description de `fct_ventes`, `table {'enforced': True, 'alias_types': True}`, ses neuf colonnes et leurs types, les deux modèles qu'il lit, `int_lignes_enrichies` et `dim_concessionnaire`, et les douze nœuds qu'il sert : l'`exposure`, `rpt_ventes_par_palier`, neuf tests de données et le test unitaire.

Vingt lignes, et un programme sait ce que le fait promet, d'où il vient et qui le lit, sans avoir ouvert l'entrepôt. Tout ce qu'il a lu, c'est vous qui l'avez écrit : retirez la description de `fct_ventes` dans `_marts.yml`, relancez les deux commandes, et la deuxième ligne sera vide. Un agent ne sait pas mieux que le projet qu'il lit. Remettez la description.

Étape 15 — La livraison

Complétez le `LIVRAISON.md` de votre dépôt d'une section sur la vérification continue : ce que la CI vérifie, sur quoi elle construit et ce qu'elle ne voit pas ; le contrat de `fct_ventes` et ses neuf colonnes ; les seuils de fraîcheur, source par source, avec leur raison ; et l'artefact de documentation. Reprenez la section finale, qui annonçait la CI comme manquante.

Puis commitez, et poussez si vous avez un dépôt sur GitHub :

```
git add LIVRAISON.md macros outils
git commit -m "Cours 08 : le menage des schemas et la lecture du manifeste"
git push
```

La poussée relance la CI une dernière fois. Elle doit passer au vert.

Récapitulatif

L'état du projet à la fin du cours :

Ce qui est en place	**Vérification**
Le linteur	`sqlfluff lint models tests snapshots` rend `All Finished!`, aucun `FAIL`
L'étoile, inchangée	`dbt build --target prod --exclude snp_concessionnaire` rend `PASS=128`, 80 618 lignes et 465 924 613,14 $ au fait des ventes
La fraîcheur	`dbt source freshness --target prod` vérifie 10 sources, 35 et 40 jours pour les mensuelles, 36 et 72 heures pour la télémétrie
Le contrat de `fct_ventes`	neuf colonnes typées ; `quantite::integer` le fait tomber

© 2026 iAlexMG. Tous droits réservés.

Ce qui est en place	Vérification
La cible de la CI	`ci/profiles.yml` versionné, `ci.duckdb` ignoré
La structure des sources	`ci/structure_sources.sql`, 27 lignes, 25 tables
La CI sur le poste	`dbt build --target ci --profiles-dir ci` rend `PASS=129` sur 130 nœuds, dont 30 objets et 3 avec des lignes
Le workflow	`.github/workflows/ci.yml`, neuf étapes, la pile épinglée dans `ci/requirements.txt`
La branche qui casse	le test unitaire la refuse sans données, `PASS=116 ERROR=1 SKIP=13`
Le ménage	`dbt run-operation nettoyer_dev` jette les trois schémas `dev_*`, et refuse `prod`
Le manifeste	31 modèles, 98 tests, 14 sources ; `fct_ventes` lit 2 nœuds et en sert 12

Le Cours 09 est le capstone. Il n'enseigne rien de neuf : il reconstruit le projet de zéro sur un entrepôt neuf, complète `LIVRAISON.md` et prouve que tout ce que les huit cours ont posé tient ensemble, la CI comprise.

Dépannage

Le message ou le symptôme	Ce qu'il veut dire	La parade
`Error: No such option '--profiles-dir'.`	l'option est passée à sqlfluff, qui ne la connaît pas	la retirer : le profil se déclare dans `.sqlfluff`
`The profile 'boreal_ae' does not have a target named 'ci'` au linteur	`profiles_dir` est rangé hors de `[sqlfluff:templater:dbt]`, et le linteur lit votre profil personnel	ranger `profiles_dir` et `target` sous `[sqlfluff:templater:dbt]`
Le linteur passe sur le poste et casse sur GitHub, « Could not find profile named 'boreal_ae' »	`profiles_dir` et `target` sont mal rangés, et votre poste compense par son profil personnel	la même : les deux clés sous `[sqlfluff:templater:dbt]`
`UninstalledPackagesFoundError` au linteur	`dbt deps` n'a pas été lancé sur cette copie	`dbt deps` avant `sqlfluff lint`
`DuplicateSectionError`	une section apparaît deux fois dans `.sqlfluff`	fusionner les deux sections

© 2026 iAlexMG. Tous droits réservés.

Le message ou le symptôme	Ce qu'il veut dire	La parade
Le snapshot à moitié décalé après `sqlfluff fix`	`template_blocks_indent = false` manque	l'ajouter sous `[sqlfluff:indentation]`, puis `git restore` le snapshot et relancer `fix`
`The following paths are ignored` sur `ci/profiles.yml`	la ligne `profiles.yml` du `.gitignore` vise tous les fichiers de ce nom	`!ci/profiles.yml`, après cette ligne
`Catalog Error: Schema with name argent already exists` au montage	`ci.duckdb` porte déjà la structure	supprimer `ci.duckdb`, puis remonter
`This model has an enforced contract that failed`	une colonne de `fct_ventes` a changé de nom, de type ou de nombre	lire le tableau : la ligne dit la colonne, le type du modèle et celui du contrat
`Binder Error: ... does not have a column named` sur la CI	une colonne mal nommée, que la CI voit même sans données	corriger le nom ; si la colonne est neuve dans la `RAW`, regénérer `ci/structure_sources.sql`
`dbt source freshness` rend `ERROR STALE` sur la télémétrie	votre dernier chargement a plus de 72 heures	c'est juste : la source est périmée. Recharger n'est pas l'objet du labo
`nettoyer_dev refuse la cible prod`	le verrou a joué	c'est voulu : la macro ne vise que les cibles de travail
`IO Error: Cannot open file ... utilisé par un autre processus`	la ligne de commande DuckDB est restée ouverte	`.exit` avant toute commande dbt

---

Annexe — La CI sur Snowflake et sur Databricks

Écrite d'après la documentation de dbt, de Databricks et de GitHub, consultée le 2026-09-23. Rien de cette annexe n'a été exécuté : aucun compte n'a servi à l'écrire, et aucun identifiant n'y figure.

Trois choses changent quand l'entrepôt est une plateforme partagée. La CI peut lire la vraie `RAW`, en lecture seule, et les tests de données y retrouvent leur sens. Elle a besoin d'un secret pour s'y connecter. Et elle doit construire dans un endroit à elle, qu'elle nettoie en partant.

Le profil de la CI reste versionné, parce qu'il ne porte toujours que des noms de variables. Sur Snowflake, la documentation de dbt annonce la fin de l'authentification par mot de passe au 31 août 2026, et recommande la paire de clés : la clé privée elle-même, en texte, passe par `private_key`.

```
boreal_ae:
  target: ci
  outputs:
```


© 2026 iAlexMG. Tous droits réservés.

```
    ci:
      type: snowflake
      account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
      user: "{{ env_var('SNOWFLAKE_USER') }}"
      private_key: "{{ env_var('SNOWFLAKE_PRIVATE_KEY') }}"
      private_key_passphrase: "{{ env_var('SNOWFLAKE_PRIVATE_KEY_PASSPHRASE') }}"
      role: CI_BOREAL
      warehouse: CI_WH
      database: BOREAL_CI
      schema: staging
      threads: 4
```

Sur Databricks, un principal de service, que Databricks recommande pour les outils automatisés plutôt que le jeton d'un utilisateur, s'authentifie par son identifiant et son secret :

```
boreal_ae:
  target: ci
  outputs:
    ci:
      type: databricks
      host: "{{ env_var('DATABRICKS_HOST') }}"
      http_path: "{{ env_var('DATABRICKS_HTTP_PATH') }}"
      auth_type: oauth
      client_id: "{{ env_var('DATABRICKS_CLIENT_ID') }}"
      client_secret: "{{ env_var('DATABRICKS_CLIENT_SECRET') }}"
      catalog: boreal_ci
      schema: staging
      threads: 4
```

Les valeurs vivent dans les secrets du dépôt GitHub, sous Settings, puis Secrets and variables, puis Actions. Le workflow les passe à l'étape par `env:`, et ne les écrit nulle part :

```
      - name: Le fond, par dbt build
        env:
          SNOWFLAKE_ACCOUNT: ${{ secrets.SNOWFLAKE_ACCOUNT }}
          SNOWFLAKE_USER: ${{ secrets.SNOWFLAKE_USER }}
          SNOWFLAKE_PRIVATE_KEY: ${{ secrets.SNOWFLAKE_PRIVATE_KEY }}
          SNOWFLAKE_PRIVATE_KEY_PASSPHRASE: ${{ secrets.SNOWFLAKE_PRIVATE_KEY_PASSPHRASE }}
        run: dbt build --target ci --profiles-dir ci
```

GitHub masque ces valeurs dans les journaux, et ne les transmet pas à un workflow déclenché depuis une copie du dépôt (*fork*) : une demande de tirage venue de l'extérieur tourne donc sans accès à l'entrepôt, et c'est voulu.

© 2026 iAlexMG. Tous droits réservés.

Quatre décisions restent à vous, et chacune a sa raison.

La décision	Ce qui la guide
Une base ou un catalogue à part pour la CI, `BOREAL_CI` ou `boreal_ci`	un rôle qui ne peut écrire que là ne peut rien abîmer ailleurs
La `RAW` lue en lecture seule, par un rôle qui n'a que ce droit	la CI vérifie le code sur les vraies données sans pouvoir les toucher
La structure vide, ou la vraie `RAW`	la vraie donne tout son sens aux tests de données, et se paie à chaque exécution
Le ménage en fin de tâche	`dbt run-operation nettoyer_dev --target ci --profiles-dir ci` dans une dernière étape, avec `if: always()`, pour qu'il tourne même après un échec

Sur une plateforme, la macro `nettoyer_dev` tourne telle quelle : `drop schema if exists … cascade` est accepté par Snowflake comme par Databricks, d'après leur documentation, et son verrou sur `prod` y vaut plus cher encore qu'en local.

© 2026 iAlexMG. Tous droits réservés.